
Data Engineering: Ingestion, Transformation and Lakehouse Design



Dr. Abdelkarim Erradi
CSE@QU

With assistance from AI tools, particularly
Microsoft Copilot and Google Gemini

Outline

1



Data Lakehouse

Source systems → Ingested data → governed, reliable, analysis-ready data

2



Data Pipeline Design Decisions

Batch vs. streaming • Push vs. pull • Full vs. incremental/CDC

3



Medallion Architecture

Bronze → Silver → Gold layers

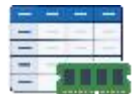
4



Data Processing using SQL

Clean, standardize, validate, and enrich

5



Data Processing using DataFrame

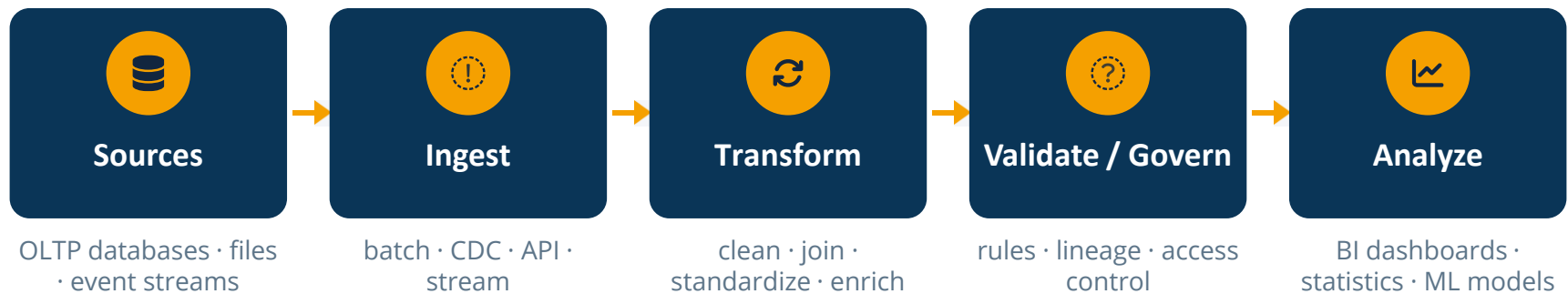
Clean, standardize, validate, and enrich



Data Lakehouse

The Data Engineering Pipeline

Data engineering is the disciplined movement of data from messy sources to trusted, analysis-ready tables.



Pipeline ground rules:

Store the raw data, then transform

- Load source data exactly as it arrives, then clean it in a later step.
- The raw copy lets you rebuild any table without re-reading the source.

Test every step, not the end

Check row counts, nulls, and unique keys after each transformation, and stop the pipeline when a check fails.

Design for reruns

Make jobs idempotent: replaying a day overwrites its rows rather than duplicating them.

OLTP Records Transactions vs. OLAP Answers Questions

	 OLTP - Operational Online Transaction Processing	 OLAP - Analytical Online Analytical Processing
Purpose:	Record day-to-day transactions.	Find patterns and answer business questions.
Typical action:	Insert or update a few rows.	Scan and summarize millions of rows
Optimized for:	Fast, reliable writes with low latency.	Joins, trends, and aggregations.
Example:	Place an order and update stock.	Compare monthly sales by region.

Why separate them?

Prevent analytical queries (full scans, joins, aggregations) from slowing down operational transactions, and vice versa

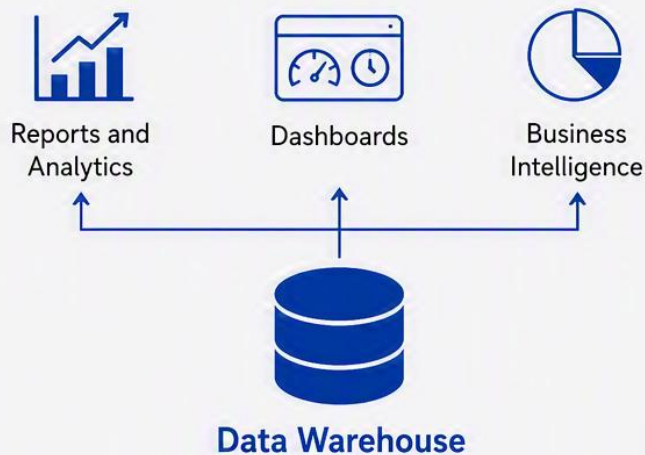


The business relies on both workloads: one supports daily operations, while the other enables analysis and insight

Data Platforms: Data Warehouse vs Data Lakehouse

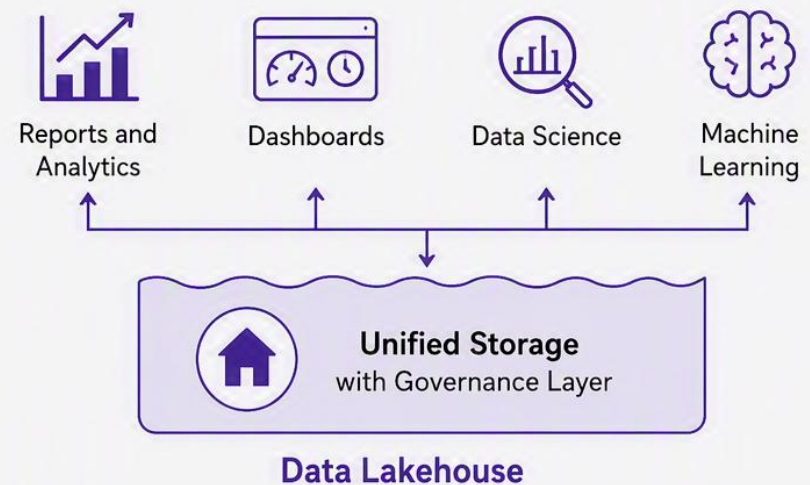
Data Warehouse

Structured data for reporting and business intelligence.



Data Lakehouse

Combines low-cost, flexible cloud storage with reliable data management, governance, and SQL/analytics capabilities in a single platform.



Storage

Proprietary database (managed storage)



Data Type

Structured



Best For

Reporting, business intelligence, dashboards



Storage

Low-cost cloud object storage with a governance layer



Data Type

Structured, semi-structured, and unstructured

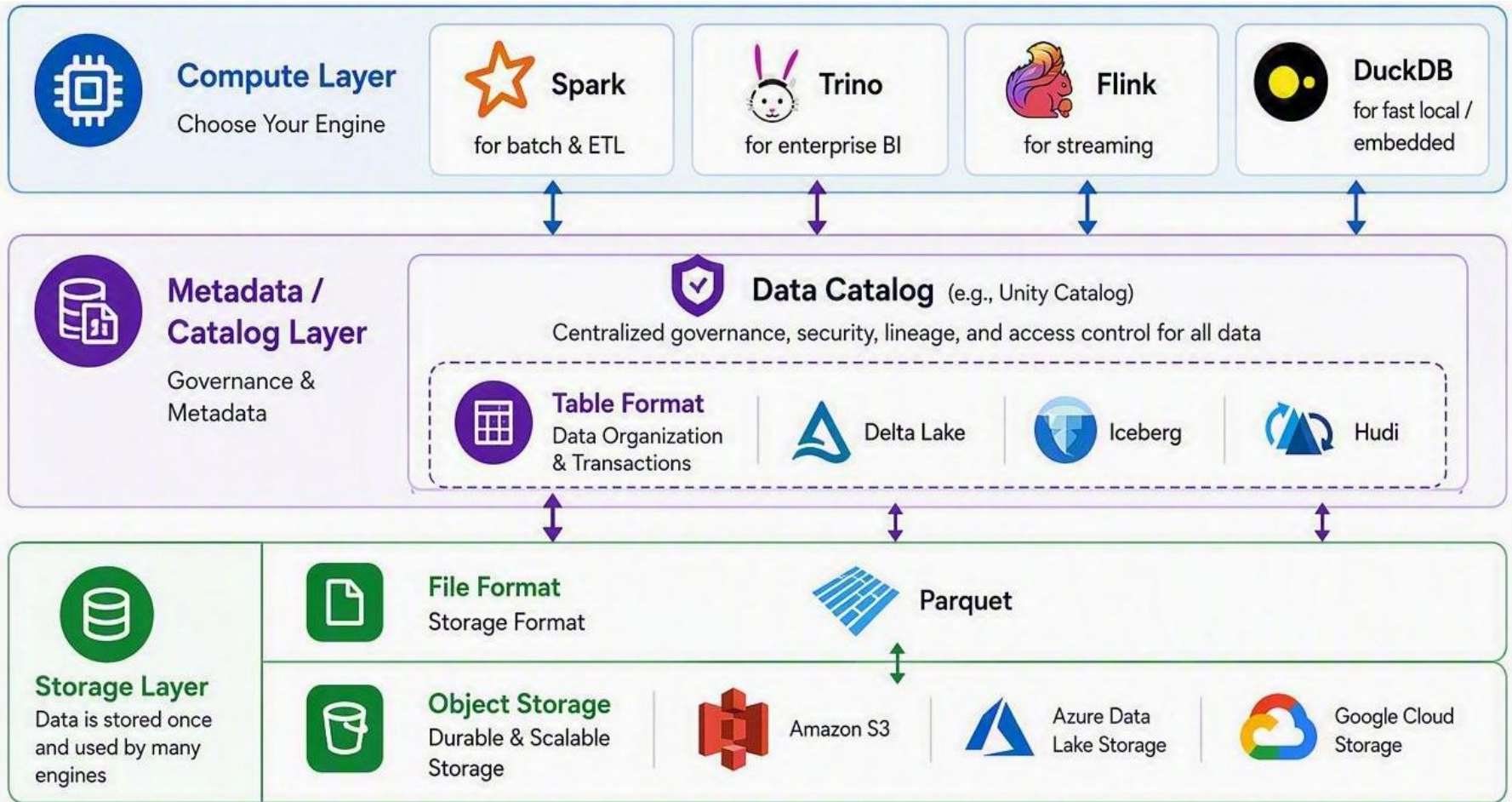


Best For

Business intelligence, data science, machine learning, and real-time analytics

Lakehouse Architecture

A Lakehouse separates where data lives (**storage**), how it is processed (**compute**), and how it is discovered and governed (**catalog**) — each layer scales independently.



Two Ideas Behind the Modern Lakehouse

Storage and Compute Are Separate

What is it

Data sits in cheap cloud storage; engines connect to it when needed

Why it helps

Add or remove processing power without copying or reloading data

In practice

Each team runs its own engine on the same data, then switches it off

Watch out

How the files are organized now decides how fast queries run

Open Table Formats

What is it

A metadata layer that turns plain files into proper tables

Why it helps

Safe updates and deletes, schema changes, and a view of older versions (time travel)

In practice

Roll back a bad load, or reproduce yesterday's numbers

Watch out

Tables need housekeeping: merge small files, clear old versions



Why they work together

- Cheap storage keeps one copy of the data; open tables make changing it safe.
- Any engine can then read the same trusted tables, without moving or copying them.

How a Lakehouse Works — Three Layers



Compute Layer

Engines that read and process the data

Start one for a job and shut it down when done; multiple engines can read the same data simultaneously.

For example: Spark, Trino, Flink or DuckDB



Metadata & Catalog

The layer that makes files behave like tables

Tracks which files form a table, its columns, its versions, and permissions.

Open table formats add [ACID](#) commits, schema evolution, and time travel

For example: a table format such as Iceberg, plus a catalog



Storage Layer

One shared copy of the data

Columnar files sitting in cloud storage — cheap, and charged separately from processing.

For example: Parquet files in S3, Azure Data Lake or Google Cloud Storage

How query runs:

Engine asks Catalog → Catalog points to files → Engine reads only those files → Result returned

Add or swap an engine anytime — the data itself never moves.

Popular Lakehouse Platforms - Four Ways to Run Analytical Workloads



Databricks

Snowflake

Microsoft Fabric

DuckDB

	Databricks	Snowflake	Microsoft Fabric	DuckDB
What it is	Clusters of machines running Spark-based pipelines	Managed engine: just write SQL	An all-in-one analytics platform	In-process engine on a single machine
Strength	Big data pipelines, streaming and ML	Fast, reliable SQL for many concurrent users	Reports and dashboards with little setup	Instant queries on files, with no setup
You pay for	Time the cluster is on	Seconds of query time; idle is free	Prepaid capacity shared by all tools	Free — your laptop
Use it when	Large-scale data engineering	Analysts need dependable SQL, not servers	Organization already using Microsoft ecosystem	Data fits on one machine; fast iteration

-  **How to choose**
- Start on one machine.
 - Add a platform when data outgrows memory or when multiple teams need governed access.

Governance keeps data secure, findable and traceable

Capability	Practical question	Online-shop example
Data cataloging	What is it, who owns it and when is it refreshed?	daily_product_sales has a definition, owner and daily refresh time.
Access control	Who can view, change or share it?	Analysts see sales; only Finance can access payment details.
Data lineage	Where did it come from and what transformed it?	Orders API → Bronze → Silver orders → daily_product_sales.
Monitoring and auditing	Is it healthy, and who used or changed it and when?	Alert on late loads; log access and permission changes.

Good governance helps people find the right data, trust it, and use it safely.



Data Pipeline Design Decisions

Pipeline Design — Five Decisions Before You Build

Agree with the source owner	What to decide	What we agreed (online orders)
1 Contact and support	Who owns the source; who resolves incidents	Commerce team owns sales data
2 Grain, schema, and key	What a single row represent (grain), the schema of each entity, and the key that uniquely identifies it	One row = one item in an order; key = order_id + line_id; 12 agreed fields
3 Change behavior	Append-only, or do records get updated and deleted?	Not append-only: orders can be updated or cancelled
4 Delivery and Ingestion frequency	How often data is delivered and how fresh it must be	Hourly incremental file. Report delay up to 1 hour is acceptable
5 Data quality and recovery	What to do with bad rows; how to replay a failed run	Orders with no ID go to quarantine; source resends on request

Choose the right data grain

Choose	Use it when	Online-orders example
Detailed grain	Supports drill-down and root-cause analysis.	Need order items for basket-level analysis. One row per order item Key = (<i>order_id</i> , <i>line_id</i>)
Summary grain	Used for KPIs, dashboards, trends, forecasting, or executive reporting	One row per day Fields = date, sales_amount, order_count, avg_order_value
Granularity Best Practice	Keep detailed and aggregated data in separate tables. Create summaries from detailed data	Order Items → Daily Sales → Monthly Sales

- **Data grain = what a single row represents**
- Define the grain before designing the table
- Keep one consistent grain per table
- Store detailed data first; create summaries separately

How data moves: batch vs streaming ingestion

Ask: how quickly a decision must be made ?

Compare	Scheduled batch (pull)	Continuous streaming (push)
Who starts it	The pipeline asks the source on a schedule	The source sends each event the moment it happens
How fresh	Minutes to hours; ready at a fixed time	Seconds-to-minutes, new events arrive continuously
How it works	One job reads and process everything changed since the last run	Each event is handled on arrival, one at a time
Online-order example	At 2 a.m. pull all of yesterday's orders for the sales database	Block a suspicious card payment before it clears
Choose it when	Historical analysis; decisions can wait	IoT monitoring; fraud detection; stock alerts; and supply chain tracking.
Watch for	Missed changes and heavy load on the source system	Duplicate and out-of-order events, and replay after an outage
Cost and effort	Cheaper, and a failed run can simply be run again	Always on, needs monitoring, alerting and on-call

Ask first: "Can the decision wait until the next scheduled run?" → Yes = batch · No = streaming

Both need a plan for failures, retries and duplicate data.

How much to load: Full, incremental or CDC

Ask: Do we reload everything, or only what changed?

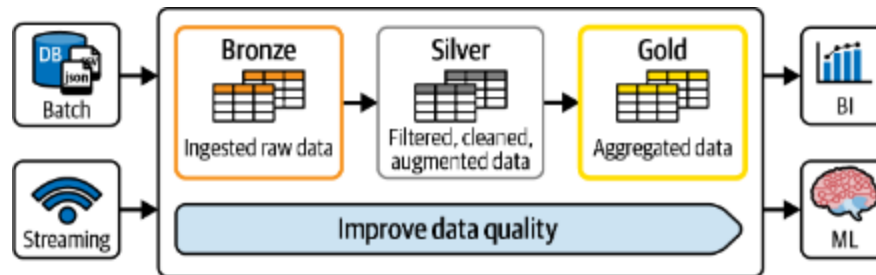
Option	What it means	Use it when	Online-order example
Full load	Copy the entire table every run	First load; or table is small	Reload full product catalogue nightly
Incremental load	Copy only new / changed rows since last run	Table too large to reload; Needs a reliable watermark (date or ID)	Load only orders placed since last night's run
Change Data Capture (CDC)	Stream every insert, update, and delete	Near-real-time sync required	Follow an order: placed → paid → cancelled

**First time do Full load then Incremental periodically.
Use CDC only when downstream must stay in sync in near real time.**

Always record what was loaded and when.



Medallion Architecture



Medallion Architecture



- A Medallion architecture arranges data into three layers, enhancing the data's structure and quality as it progresses through the layers

Bronze → Silver → Gold: data gets cleaner at each stage

 Bronze	 Silver	 Gold
• Store exactly as received — never edit • Add tracking metadata: <code>source_name</code> · <code>ingested_at</code> · <code>load_id</code> • E.g., Raw order files from each store	• Fix types · handle nulls · remove duplicates • Standardize values · apply business rules • Enrich rows with reference data joins • E.g., One clean orders table, one row per order line	• Aggregate to the right grain · derive business metrics • Model facts and dimensions · rename for the business • One definition per metric — build only on Silver • E.g., Daily sales by product, ready for the dashboard
 <i>If processing fails, replay from Bronze — without asking the source to resend</i>	 <i>Never delete a bad row silently — quarantine, reject, or flag it</i>	 <i>Rebuild, never patch — check totals every run</i>

3 Bronze: the row data plus tracking columns

Ingestion details added to every row such as:	Online-order example	Why it matters
source_name	orders_api	Says which system the row came from
ingested_at	2026-09-03 10:03	Shows when it arrived -> can help identify late data
load_id	batch_2026_09_03_01	Lets you replay just one bad run

Write Bronze once, never edit it. Fix data in Silver, so you can always rebuild.



Silver Layer Data Transformations

16

Transformation	What it means	Online-order example
Fix data types	Text becomes numbers, dates and Booleans	"19.99" becomes 19.99; "2026-09-03" becomes a date
Handle missing values	Fill, flag, or quarantine blanks	Order with no customer_id goes to quarantine
Remove duplicates	Keep one row per unique key	Same order sent twice by the API: keep the latest
Standardise values	One spelling, one unit, one code, one format	"QA", "Qatar", "qatar" all become QAT; dates stored as YYYY-MM-DD
Apply business rules	Reject rows that cannot be true	quantity <= 0 or a future order_date is rejected
Integrate reference data	Connect each row to its related data	Order joined to its customer; product_id 88 gains its name
Enrich with new fields	Add a useful value calculated from existing fields	delivery_days = delivered_at – order_date

**Silver is where messy data becomes trusted, analytics-ready data.
Not every dataset requires every transformation step.**

See more details
in slides 03



Gold Layer Data Transformations

18

Transformation	What it means	Online-order example
Aggregate to a grain	Roll many rows up to the level the business asks about	Aggregate individual order lines into units sold and revenue per product, per store, per day.
Derive business metrics	Turn raw columns into the agreed measures	$\text{revenue} = \text{quantity} \times \text{unit_price} - \text{discount}$
Model facts and dimensions	One table of events joined to descriptive tables	fact_sales joined to dim_product, dim_store and dim_date
Add time comparisons	Pre-compute the periods people always ask for	Month-to-date revenue and the same day last year
Rename for the business	Technical names become words users recognise	qty_ord becomes Units sold; amt_net becomes Revenue
Filter to the audience	Publish only the rows and columns a user may see	A store manager sees only their own store's rows

Gold is where trusted data becomes business-ready insights that people can act on.

See more details
in slides 04



Data Processing using SQL



SQL — The Working Language of Data Engineering

SQL is how data engineers talk to data. It is a declarative language that enable you describe **WHAT** results you want, and the query engine works out how to produce it.
Its commands fall into four families:

Family	Commands you will meet	What it is for
DDL Data Definition Language	CREATE, ALTER, DROP	Define the structure: tables, columns, schemas
DML Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Read the data and change what is stored in it
DCL Data Control Language	GRANT, REVOKE	Control who may read or change each table
TCL Transaction Control Language	COMMIT, ROLLBACK	Group changes so they all succeed, or none do

```
-- DDL: define the table once
CREATE TABLE silver.orders (...);
-- DML: Load Bronze into Silver every day
INSERT INTO silver.orders
  SELECT * FROM bronze.orders;
-- DCL + TCL: grant access, commit safely
GRANT SELECT ON silver.orders TO analyst;
```



SELECT — One Statement, Four Jobs

	What it does	Example
SELECT	Choose or compute the columns returned	<code>SELECT order_id, total * 1.05 AS with_tax</code>
WHERE	Keep only the rows that qualify	<code>WHERE status = 'shipped'</code>
GROUP BY	Collapse many rows into one per group	<code>GROUP BY customer_id</code>
ORDER BY / LIMIT	Sort the result and cap how much comes back	<code>ORDER BY revenue DESC LIMIT 10</code>

```
-- Top 5 customers by September revenue
SELECT customer_id,
       SUM(order_total) AS revenue
FROM   sales.orders
WHERE  order_date >= '2026-09-01'
GROUP BY customer_id
ORDER BY revenue DESC LIMIT 5;
```



Key idea

- One statement answers four questions: which rows, which columns, grouped how, sorted how.



SELECT and WHERE — Pick Columns, Filter Rows

	What it does	Example on sales.orders
= > < >=	Compare a column to a value	WHERE order_total >= 100
BETWEEN / IN	Match a range or a short list	WHERE order_date BETWEEN '2026-09-01' AND '2026-09-30'
LIKE	Match a text pattern	WHERE email LIKE '%@gmail.com'
IS NULL	Find rows with a missing value	WHERE shipped_at IS NULL

```
-- September shipped orders over 500 QR
SELECT order_id, customer_id, order_total
FROM sales.orders
WHERE order_date >= '2026-09-01'
      AND status = 'shipped'
      AND order_total > 500;
```

⚠ Common traps

- NULL means unknown: status = NULL matches nothing. Use IS NULL.
- SELECT * breaks when columns change — name the columns you need.

SQL Runs in a Logical Order — Not the Order You Write

Logical execution order

1 FROM — pick the tables



2 WHERE — filter rows



3 GROUP BY — build groups



4 HAVING — filter groups



5 SELECT — choose columns



6 ORDER BY — sort output

The order you write it

```
SELECT country, COUNT(*) AS orders
FROM sales.orders
WHERE order_date >= '2026-01-01'
GROUP BY country
HAVING COUNT(*) > 100
ORDER BY orders DESC
LIMIT 10;
```

Why it matters: WHERE filters rows before grouping; HAVING filters after



Joins Combine Tables

	What you get	Use it when
INNER	Only rows matching in both tables	Every order must have a valid customer
LEFT	All left rows; NULLs where no match	Keep all customers, even those with no orders
RIGHT	All right rows; NULLs where no match	Rarely used — rewrite as LEFT for readability

```
SELECT c.customer_name,  
       COUNT(o.order_id) AS orders  
FROM sales.customers AS c  
LEFT JOIN sales.orders AS o  
      ON o.customer_id = c.customer_id  
GROUP BY c.customer_name;
```

⚠ Common trap

- A LEFT JOIN plus a WHERE filter on the right table silently becomes an INNER JOIN.
- Put that condition in the ON clause instead.



Aggregation Turns Raw Rows Into Business Metrics

BEFORE — raw order rows

order_id	country	amount
1001	Qatar	120.00
1002	Qatar	80.00
1003	UAE	200.00
1004	UAE	50.00
1005	Qatar	45.00

The query

```
SELECT country,  
       COUNT(*) AS orders,  
       SUM(amount) AS revenue,  
       AVG(amount) AS avg_order  
FROM sales.orders  
GROUP BY country  
HAVING SUM(amount) > 100  
ORDER BY revenue DESC;
```

AFTER — one row per group

country	orders	revenue	avg_order
Qatar	3	245.00	81.67
UAE	2	250.00	125.00



Important Notes

- COUNT(*) counts rows; COUNT(col) skips NULLs.
- Every non-aggregated SELECT column must appear in GROUP BY.
- HAVING filters groups; WHERE filters rows before grouping.



CTEs Turn Sub Query Into Readable Steps

BEFORE — nested subquery

```
SELECT customer_id, COUNT(*) AS cnt
FROM (
  SELECT *
  FROM sales.orders
  WHERE order_date >=
    CURRENT_DATE - INTERVAL '30' DAY
) AS t
GROUP BY customer_id;
```

AFTER — named CTE

```
WITH recent_orders AS (
  SELECT *
  FROM sales.orders
  WHERE order_date >=
    CURRENT_DATE - INTERVAL '30' DAY
)
SELECT customer_id, COUNT(*) AS cnt
FROM recent_orders
GROUP BY customer_id;
```



Readable each step gets a name



Reusable reference a CTE more than once



Debuggable run one step at a time



Key idea

A Common Table Expression (CTE) is a name for a result set, not a stored table — it disappears when the query ends.



INSERT, UPDATE, DELETE — Changing Data Safely

	What it does	Example on silver.orders
INSERT	Add new rows to a table	<code>INSERT INTO silver.orders SELECT ... FROM bronze.orders</code>
UPDATE	Change values in rows that already exist	<code>UPDATE silver.orders SET status = 'delivered'</code>
DELETE	Remove the rows that match a condition	<code>DELETE FROM silver.orders WHERE order_id = 1042</code>
MERGE	Update the row if it exists, insert it if it does not	<code>MERGE INTO silver.orders USING bronze.orders ON order_id ...</code>

```
-- INSERT: append one day of orders
INSERT INTO silver.orders
  SELECT * FROM bronze.orders
  WHERE order_date = '2026-09-11';

-- MERGE: the rerun-safe version of that load
MERGE INTO silver.orders AS s
  USING bronze.orders AS b ON s.order_id = b.order_id
  WHEN MATCHED THEN UPDATE SET s.status = b.status
  WHEN NOT MATCHED THEN INSERT (order_id, status)
    VALUES (b.order_id, b.status);
```



Common traps

- **Prevent full-table updates:** A missing clause can update every row. Test the WHERE clause with a SELECT before running an UPDATE or DELETE.
- **Ensure safe reruns (idempotent loads):** Use MERGE with a reliable key to update matches, insert new rows, and prevent duplicates.



SQL Best Practices

	Why it matters	Do this
Name your columns	A new upstream column silently changes what SELECT * returns	SELECT order_id, status -- not SELECT *
Filter early	Rows dropped before a join never have to be moved or matched	WHERE order_date >= '2026-09-01'
Qualify and alias	Bare table names break when the same name exists in two schemas	FROM silver.orders AS o
Handle NULLs on purpose	Unknown values quietly drop rows and skew every average	COALESCE(discount, 0); IS NOT NULL

```
-- readable, filtered early, NULL-safe
SELECT o.order_id,
       c.customer_name,
       COALESCE(o.discount, 0) AS discount
FROM   silver.orders      AS o
JOIN   silver.customers  AS c
ON     c.customer_id = o.customer_id
WHERE  o.order_date >= '2026-09-01'
AND    o.status = 'shipped';
```

⚠ Common traps

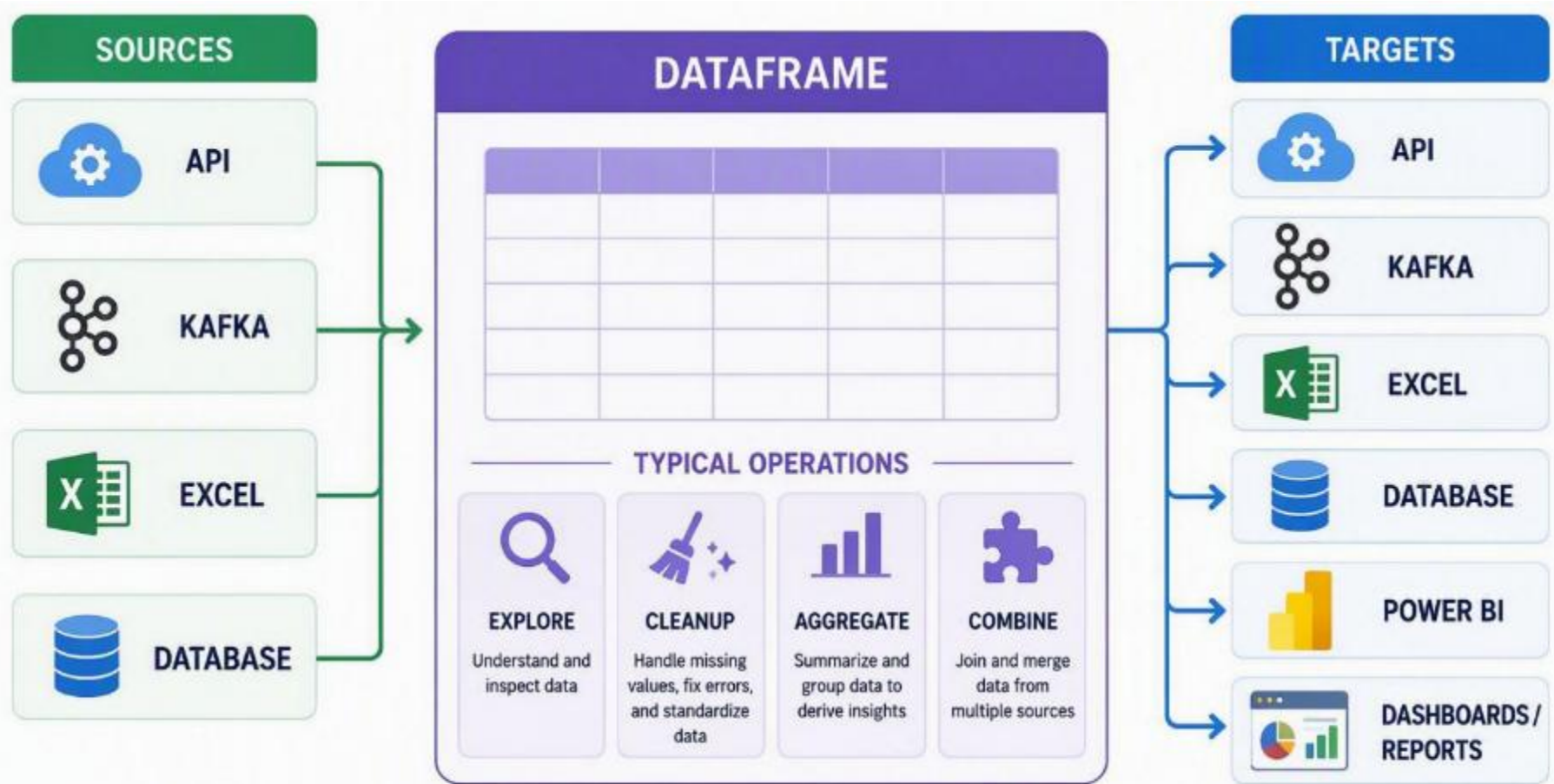
WHERE vs HAVING: filter rows in WHERE; use HAVING only for aggregates like COUNT(*) > 5.

SELECT * in a pipeline: one new upstream column can silently break tomorrow's load.



Data Processing using DataFrame

DataFrame = Core of ETL and Analytics



ONE STRUCTURE. MANY PURPOSES. DataFrame connects diverse sources to multiple targets through consistent operations—powering both data engineering and analytics.

Pandas, Polars, PySpark: Same DataFrame, Different Engine

	 pandas	 Polars	 PySpark
Best for	Exploration, small ad-hoc analysis	Mid-size data on one machine, CPU-heavy work	Large-scale ETL pipelines, complex joins, streaming
Sweet spot	< 5M rows, < 500 MB files	5M–50M rows, multi-GB CSV / Parquet	Datasets too large for one machine; 100M+ rows
Strength	Simple, flexible, huge ecosystem	Multi-core processing, lazy evaluation, and a columnar memory format for fast processing	Distributed processing, scalable transformations
Execution	Eager — runs each step right away	Lazy — optimizes the whole query first	Lazy — plans first, then runs it efficiently
Limit	Single-threaded, memory-bound, eager only	Single machine — not distributed	Cluster overhead on small jobs; costly infrastructure to set up

Rule of thumb: start on the smallest engine that fits your data; move up only when size or governance forces it.

On 10M rows: pandas ~40s • Polars ~5s • Spark 8–12s locally, but scales past 100M rows.



Read & Filter - The SELECT and WHERE of DataFrame

	SQL clause	Pandas equivalent on df
SELECT cols	Pick a subset of columns	<code>df[["order_id", "order_total"]]</code>
WHERE cond	Keep rows that match a mask	<code>df[df.order_total > 500]</code>
IN (list)	Match any value in a list	<code>df[df.status.isin(["shipped", "paid"])]</code>
SELECT + WHERE	Filter rows and columns at once	<code>df.loc[df.order_total > 500, ["order_id", "order_total"]]</code>

```
# September shipped orders over 500 QR
df = pd.read_parquet("bronze/orders")
cols = ["order_id", "customer_id",
"order_total"]
m = (df.status == "shipped") & \
    (df.order_total > 500) & \
    (df.order_date >= "2026-09-01")
out = df.loc[m, cols]
```

⚠ Common traps

- **Combining filters:** Use **&** and **|**, with each condition in parentheses.

```
df[(df["sales"] > 1000) &
(df["region"] == "Qatar")]
```

- Use `.loc[mask, columns]` for safe filtering and assignment.

```
df.loc[(df["sales"] > 1000) & (df["region"]
== "Qatar"), "priority"] = "High"
```



pd.merge — Joining DataFrames, the SQL JOIN

	SQL JOIN	pd.merge equivalent
INNER JOIN	Keep only matching keys	how="inner" (default)
LEFT JOIN	All orders, matched customers	how="left"
RIGHT JOIN	All customers, matched orders	how="right"
FULL OUTER JOIN	Keep every row from both	how="outer"

```
# Attach the customer to each order
orders = pd.read_parquet("bronze/orders")
customers =
pd.read_parquet("bronze/customers")
df = pd.merge(orders, customers,
              on="customer_id",
              how="left")
```

⚠ Common traps

- One-to-many keys multiply rows: if a customer_id repeats, each order matches it more than once, so rows quietly duplicate. Add validate="m:1" to catch it.
- Same-named columns clash — e.g. two total columns. Keep both with suffixes=("_ord", "_cust"), giving total_ord and total_cust.



groupby & agg — The GROUP BY of DataFrames

	SQL	Pandas equivalent
GROUP BY customer_id	Build one group per customer	<code>df.groupby("customer_id")</code>
SUM(order_total)	Total revenue per group	<code>.agg(revenue=("order_total", "sum"))</code>
COUNT(*)	Number of orders per group	<code>.agg(n_orders=("order_id", "count"))</code>
HAVING SUM(...) > 1000	Filter groups after aggregating	<code>g[g.revenue > 1000]</code>

```
# Revenue per customer for September
g = (df.groupby("customer_id")
     .agg(
         revenue=("order_total", "sum"),
         n_orders=("order_id", "count"))
     .reset_index())
```

⚠ Common traps

- HAVING has no direct method: aggregate first, then filter the result with a boolean mask.
- agg skips NaN by default, so sum and count ignore missing values



Transform & Upsert — Writing bronze into silver

	What it does	Pandas on df / silver
Add column	Derive a new column from existing ones	<code>df.assign(with_tax=df.order_total * 1.05)</code>
Update rows	Change values where a mask is true	<code>df.loc[df.order_id == 1042, "status"] = "delivered"</code>
Remove	Drop columns (or rows) you no longer need	<code>df.drop(columns=["_ingest_tmp"])</code>
Upsert	Update matches and append new rows, rerun-safe	<code>pd.concat([silver, df]).drop_duplicates("order_id", keep="last")</code>

```
# Add, update and remove columns/rows
df = df.assign(with_tax=df.order_total * 1.05)
df.loc[df.order_id == 1042, "status"] = "delivered"
df = df.drop(columns=["_ingest_tmp"])

# Upsert: the rerun-safe write into silver
silver = pd.read_parquet("silver/orders")
combined = pd.concat([silver, df])
silver = combined.drop_duplicates("order_id", keep="last")
silver.to_parquet("silver/orders")
```



Common traps

- **assign and drop return copies:** they do not change df in place, so reassign the result (df = df.assign(...)) or your edit is lost.
- **Make the upsert idempotent:** concat then drop_duplicates on the key with keep=last, so a rerun overwrites instead of doubling rows.



Clean & Missing Data — Preparing bronze for silver

	Cleaning task	Pandas method on df
Dedupe	Drop repeated order_id rows	<code>df.drop_duplicates("order_id")</code>
Drop missing	Remove rows with no key	<code>df.dropna(subset=["order_id"])</code>
Fill missing	Replace NaN with a default	<code>df.status.fillna("unknown")</code>
Cast types	Enforce the silver schema	<code>df.order_total.astype("float64")</code>

```
# Tidy bronze orders before the load
df = df.drop_duplicates("order_id")
df = df.dropna(subset=["order_id"])
df["status"] = df.status.fillna("unknown")
df["order_total"] =
df.order_total.astype("float64")
df["order_date"] =
pd.to_datetime(df.order_date)
```

⚠ Common traps

- `fillna(0)` on numbers hides missing values and skews sums and averages — fill deliberately.
- `astype("int64")` fails if NaN remains; use the nullable `"Int64"` dtype instead.

Process columns as a pipeline - not row by row

Pipeline = chain of operations on orders: filter → clean → derive → group → summarize → sort.

```
summary = (orders
    .loc[orders["status"] == "Complete"]
    .dropna(subset=["amount"])
    .drop_duplicates("order_id")
    .assign(net_amount=lambda d:
        d["amount"] * 0.95)
    .groupby("channel", as_index=False)
    .agg(orders=("order_id", "nunique"),
        revenue=("net_amount", "sum"))
    .sort_values("revenue", ascending=False)
)
```

Read the chain

- 1 Keep complete orders
- 2 Handle missing + duplicates
- 3 Create net_amount
- 4 Group by channel
- 5 Count orders + sum revenue
- 6 Put highest revenue first